Phase 2 Day 2 – ESP32 WiFi Foundation and Status Alert Testing

1. Objective

The objective of **Phase 2 Day 2** was to test the WiFi capability of the ESP32 and prepare it for future IoT-based alerting.

After completing Phase 2 Day 1, the ESP32 hardware foundation was ready. The board was detected in Arduino IDE, external LED and buzzer were tested, and Serial Monitor command mode was verified.

On Day 2, the main goal was to confirm that the ESP32 can:

```
Connect to a WiFi network
Display WiFi status in Serial Monitor
Print its local IP address
Use LED as WiFi status indicator
Use buzzer as success/failure indicator
Handle WiFi failure condition properly
```

This step is important before adding Telegram alerts or cloud-based IoT notifications.

---

2. Work Completed

The following tasks were completed on Phase 2 Day 2:

```
Created Phase 2 Day 2 project folder structure
Tested basic ESP32 WiFi connection
Verified ESP32 IP address output in Serial Monitor
Tested WiFi status indication using LED and buzzer
Tested WiFi failure condition using wrong password
Saved Serial Command Mode as backup code
Prepared Day 2 report notes
```

---

3. Hardware Components Used

Component	Purpose
ESP32 WiFi + Bluetooth Board	Main IoT microcontroller
External LED	Visual WiFi status indicator
220Ω Resistor	Current limiting for LED
Active Buzzer	Audio indication for success/failure
Breadboard	Temporary circuit connection
Jumper Wires	Wiring connections
USB Cable	Programming and power supply

---

4. Software and Tools Used

Software / Tool	Purpose
Arduino IDE	Programming ESP32
ESP32 Board Support Package	ESP32 board programming support
WiFi.h Library	ESP32 WiFi connectivity
Serial Monitor	WiFi status and IP address monitoring
Silicon Labs USB to UART Bridge VCP Driver	ESP32 COM port detection

---

5. Final Pin Configuration

The pin configuration used from Day 1 was continued on Day 2.

```
LED Pin    → GPIO 23
Buzzer Pin → GPIO 22
GND        → Common Ground
Baud Rate  → 9600
Board      → ESP32 Dev Module
```

This configuration was used because GPIO 23 worked reliably for the external LED during Day 1 testing.

---

6. Folder Structure Created

A separate Day 2 folder structure was prepared to keep the project organized.

Suggested structure:

```
Phase-2-ESP32-IoT-Alerting/
│
├── Day-1-ESP32-Setup/
│
├── Day-2-WiFi-Foundation/
│   ├── esp32_wifi_test/
│   │   └── esp32_wifi_test.ino
│   │
│   ├── esp32_wifi_led_buzzer_status/
│   │   └── esp32_wifi_led_buzzer_status.ino
│   │
│   └── notes/
│       └── phase2_day2_notes.md
│
└── backup_codes/
    └── esp32_command_mode.ino
```

This structure helps separate basic setup, WiFi testing, status indication testing, and backup codes.

---

7. Task 1 – Project Folder Setup

The first task was to create and organize the Phase 2 Day 2 files.

The following folders were prepared:

```
Day-2-WiFi-Foundation
esp32_wifi_test
esp32_wifi_led_buzzer_status
notes
backup_codes
```

This made the project easier to manage and prepared it for future IoT alerting code.

Result

```
Task 1 completed successfully.
Day 2 project folder structure was created.
```

---

8. Task 2 – ESP32 WiFi Connection Test

The second task was to connect the ESP32 to a WiFi network using the built-in ESP32 WiFi library.

The `WiFi.h` library was used.

Code Used

```
#include <WiFi.h>

const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";

void setup() {
  Serial.begin(9600);
  delay(1000);

  Serial.println("Connecting to WiFi...");
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("");
  Serial.println("WiFi Connected Successfully!");
  Serial.print("ESP32 IP Address: ");
  Serial.println(WiFi.localIP());
}

void loop() {
}
```

The WiFi name and password were entered in the code before uploading.

Serial Monitor Settings

```
Baud Rate: 9600
```

Expected Output

```
Connecting to WiFi...
.....
WiFi Connected Successfully!
ESP32 IP Address: 192.168.x.x
```

Observed Result

The ESP32 connected successfully to the WiFi network and printed the local IP address in the Serial Monitor.

Result

```
Task 2 completed successfully.
ESP32 WiFi connection was verified.
```

---

9. Task 3 – WiFi Status Indication Using LED and Buzzer

After confirming basic WiFi connection, the next task was to add physical status indication using the external LED and buzzer.

The purpose was to make the ESP32 show connection progress and result without depending only on the Serial Monitor.

Status Logic

Condition	LED Output	Buzzer Output
Connecting to WiFi	LED blinks	OFF
WiFi connected successfully	LED stays ON	One short beep
WiFi connection failed	LED stops	Three short beeps

Code Used

```
#include <WiFi.h>

#define LED_PIN 23
#define BUZZER_PIN 22

const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";

void setup() {
  Serial.begin(9600);

  pinMode(LED_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  digitalWrite(LED_PIN, LOW);
  digitalWrite(BUZZER_PIN, LOW);

  Serial.println("Connecting to WiFi...");
  WiFi.begin(ssid, password);

  int attempt = 0;

  while (WiFi.status() != WL_CONNECTED && attempt < 20) {
    digitalWrite(LED_PIN, HIGH);
    delay(200);
    digitalWrite(LED_PIN, LOW);
    delay(300);
```

```
    Serial.print(".");
    attempt++;
  }

  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("");
    Serial.println("WiFi Connected Successfully!");
    Serial.print("ESP32 IP Address: ");
    Serial.println(WiFi.localIP());

    digitalWrite(LED_PIN, HIGH);

    digitalWrite(BUZZER_PIN, HIGH);
    delay(300);
    digitalWrite(BUZZER_PIN, LOW);
  }
  else {
    Serial.println("");
    Serial.println("WiFi Connection Failed!");

    for (int i = 0; i < 3; i++) {
      digitalWrite(BUZZER_PIN, HIGH);
      delay(200);
      digitalWrite(BUZZER_PIN, LOW);
      delay(200);
    }
  }
}

void loop() {
}
```

Observed Result

The LED blinked while the ESP32 was attempting to connect to WiFi.
After successful connection, the LED stayed ON and the buzzer gave one short beep.

The Serial Monitor displayed the WiFi connection success message and the ESP32 local IP address.

Result

```
Task 3 completed successfully.
LED and buzzer WiFi status indication worked correctly.
```

---

10. Task 4 – WiFi Failure Case Testing

The fourth task was to test how the ESP32 behaves when WiFi connection fails.

This was done by intentionally entering a wrong WiFi password.

Purpose of Failure Test

This test was important because IoT systems should not only work during success conditions. They must also give clear feedback during failure conditions.

A failure condition may happen due to:

```
Wrong WiFi password
Router unavailable
Weak WiFi signal
Network disconnected
SSID changed
```

Test Method

The WiFi password in the code was temporarily changed to an incorrect password.

Example:

```
const char* password = "WRONG_PASSWORD";
```

The code was uploaded again and Serial Monitor was opened at 9600 baud rate.

Expected Output

```
Connecting to WiFi...
....................
WiFi Connection Failed!
```

Expected Hardware Output

```
LED blinks during WiFi connection attempts
Buzzer beeps three times after connection failure
```

Observed Result

The ESP32 attempted to connect to WiFi.
After the connection attempt limit was reached, the Serial Monitor displayed the failure message.
The buzzer beeped three times to indicate failure.

After the failure test, the correct WiFi password was restored and the ESP32 connected successfully again.

Result

```
Task 4 completed successfully.
WiFi failure condition was tested and verified.
```

---

11. Task 5 – Serial Command Mode Backup

The Day 1 Serial Command Mode code was saved as a backup.

Backup file:

```
esp32_command_mode.ino
```

Why Backup Was Important

This backup code is useful for quick hardware testing if future WiFi or Telegram code creates issues.

It can verify:

```
ESP32 upload is working
Serial communication is working
LED is working
Buzzer is working
GPIO pins are working
```

Backup Code Reference

```
#define LED_PIN 23
#define BUZZER_PIN 22

void setup() {
  pinMode(LED_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  Serial.begin(9600);
}

void loop() {
  if (Serial.available() > 0) {
    char data = Serial.read();

    if (data == '1') {
      digitalWrite(LED_PIN, HIGH);
      digitalWrite(BUZZER_PIN, HIGH);
    }
    else if (data == '0') {
      digitalWrite(LED_PIN, LOW);
      digitalWrite(BUZZER_PIN, LOW);
    }
  }
}
```

Result

```
Task 5 completed successfully.
Serial Command Mode backup was saved.
```

12. Challenges Faced and Solutions

Challenge	Reason	Solution	Status
Needed WiFi connection verification	ESP32 IoT alerting requires stable WiFi first	Tested WiFi using `WiFi.h` library	Solved
Needed physical WiFi status indication	Serial Monitor alone is not enough for hardware feedback	Used LED and buzzer for connection status	Solved
Needed failure condition testing	IoT systems must handle connection failure	Tested wrong password case	Solved
Needed hardware fallback code	Future IoT code may create debugging difficulty	Saved Serial Command Mode as backup	Solved

13. Final Output Summary

At the end of Phase 2 Day 2, the ESP32 successfully performed the following:

```
Connected to WiFi
Printed local IP address
Displayed connection progress in Serial Monitor
Blinked LED during connection attempt
Kept LED ON after successful connection
Beeped buzzer once after WiFi success
Detected WiFi failure condition
Beeped buzzer three times after WiFi failure
Reconnected successfully after restoring correct password
```

---

14. Final Working Configuration

```
Board: ESP32 Dev Module
Port: COM4
WiFi Library: WiFi.h
Baud Rate: 9600
LED Pin: GPIO 23
Buzzer Pin: GPIO 22
Status LED: Active during WiFi status
Buzzer: Success/failure audio indicator
```

---

15. Importance of Day 2

Phase 2 Day 2 was an important step because it moved the ESP32 from simple local output control to network connectivity.

Day 1 proved:

```
ESP32 can be programmed
LED and buzzer can be controlled
Serial command mode works
```

Day 2 proved:

```
ESP32 can connect to WiFi
ESP32 can report network status
ESP32 can indicate success and failure physically
ESP32 is ready for IoT alert integration
```

This prepares the system for the next stage, where WiFi-based notifications can be implemented.